

INFORME PROYECTO SO – PARTE 2

SIMULACIÓN DE ATAQUES, REGISTRO DE EVENTOS Y ANÁLISIS DE CONCURRENCIA EN POSTGRESQL

Estudiantes

Jhoan Hurtado Marin – 202459472

Juan Stevan Cruz – 202459437

Asignatura

Sistemas Operativos

1. Resumen

En esta segunda fase del proyecto se ampliaron las funcionalidades desarrolladas previamente incorporando mecanismos para generación de carga concurrente, simulación de ataques HTTP Flood, almacenamiento de eventos de ejecución y análisis de concurrencia dentro de PostgreSQL.

El objetivo principal fue evaluar el comportamiento de la aplicación web y de la base de datos bajo distintos escenarios de estrés, observando el consumo de recursos, la capacidad de recuperación del sistema y el funcionamiento de los mecanismos de sincronización implementados por PostgreSQL.

Las pruebas realizadas permitieron identificar el impacto de diferentes niveles de carga sobre CPU, memoria RAM y procesos activos. Asimismo, se comprobó experimentalmente el funcionamiento de los bloqueos de transacciones mediante escenarios de Lock Contention, observando cómo PostgreSQL administra el acceso concurrente a los recursos compartidos.

2. Introducción

Los sistemas modernos deben ser capaces de responder correctamente ante múltiples solicitudes concurrentes, manteniendo la integridad de la información y garantizando un rendimiento aceptable incluso bajo condiciones de alta carga.

En entornos reales, aplicaciones web, servicios empresariales y plataformas distribuidas reciben continuamente solicitudes simultáneas que pueden generar cuellos de botella, conflictos de acceso y consumo elevado de recursos.

Con el fin de estudiar estos fenómenos, se implementó un conjunto de mecanismos de prueba sobre una aplicación desarrollada con Next.js y una base de datos PostgreSQL ejecutada en contenedores Docker. Las pruebas incluyeron simulación de ataques HTTP Flood, registro de eventos de carga, monitoreo de recursos y análisis de bloqueos entre transacciones concurrentes.

3. Objetivos

Objetivo General

Evaluar el comportamiento de PostgreSQL y de la aplicación web bajo condiciones de carga concurrente mediante pruebas de estrés, monitoreo de recursos y análisis de mecanismos de sincronización.

Objetivos Específicos

- Implementar un módulo HTTP Flood para generar carga concurrente.
 - Registrar eventos de ejecución dentro de la base de datos.
 - Analizar el comportamiento de CPU y memoria RAM durante las pruebas.
 - Evaluar la capacidad de recuperación del sistema tras situaciones de alta carga.
 - Comprobar experimentalmente el funcionamiento de los bloqueos y mecanismos de concurrencia de PostgreSQL.
-

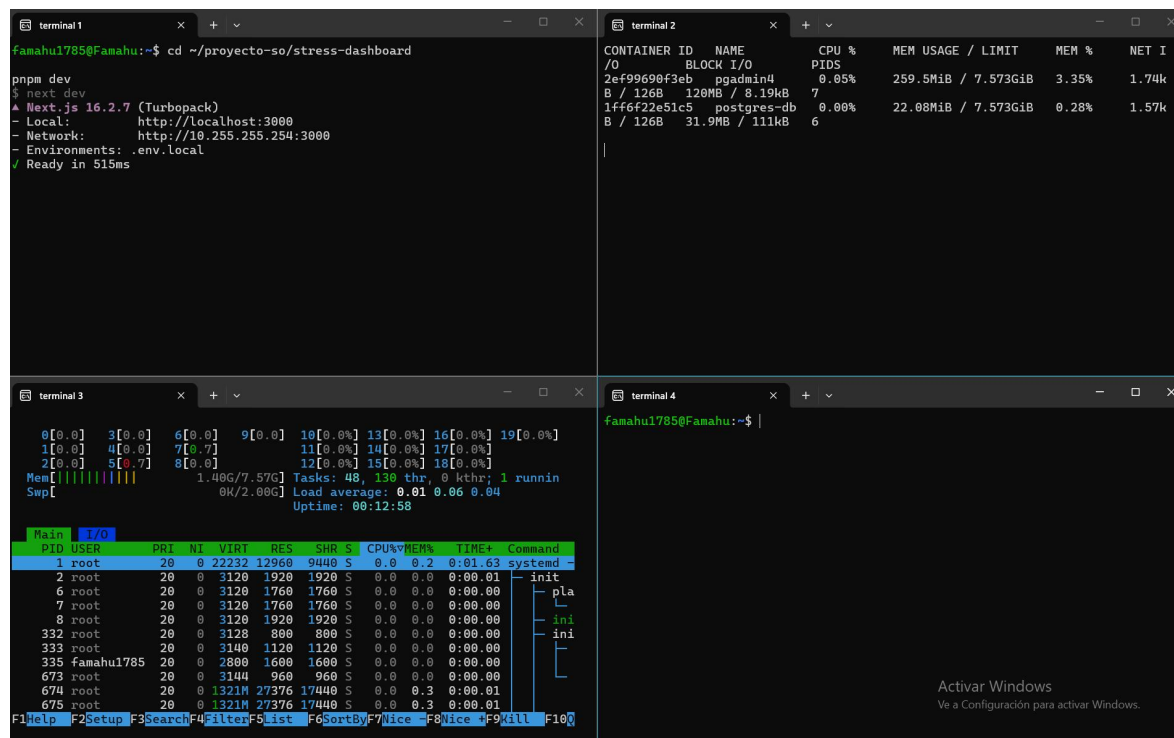
4. Estado Inicial del Sistema

Antes de iniciar las pruebas se verificó el correcto funcionamiento de todos los componentes involucrados en el proyecto.

La aplicación Next.js se encontraba ejecutándose correctamente en el puerto 3000, mientras que PostgreSQL y PgAdmin operaban dentro de contenedores Docker.

El monitoreo mediante Docker Stats y htop mostró un consumo mínimo de recursos, indicando que el sistema estaba preparado para iniciar las pruebas experimentales.

Figura 1. Estado inicial del sistema.



Análisis

En esta captura puede observarse que la aplicación web se encuentra operativa y que los contenedores PostgreSQL y PgAdmin están funcionando correctamente. El uso de CPU es prácticamente nulo y el consumo de memoria se mantiene dentro de valores reducidos. Esta condición representa la línea base utilizada para comparar los resultados obtenidos durante las pruebas de estrés.

5. Implementación del Módulo HTTP Flood

Con el propósito de generar carga artificial sobre el sistema se desarrolló un nuevo endpoint denominado HTTP Flood.

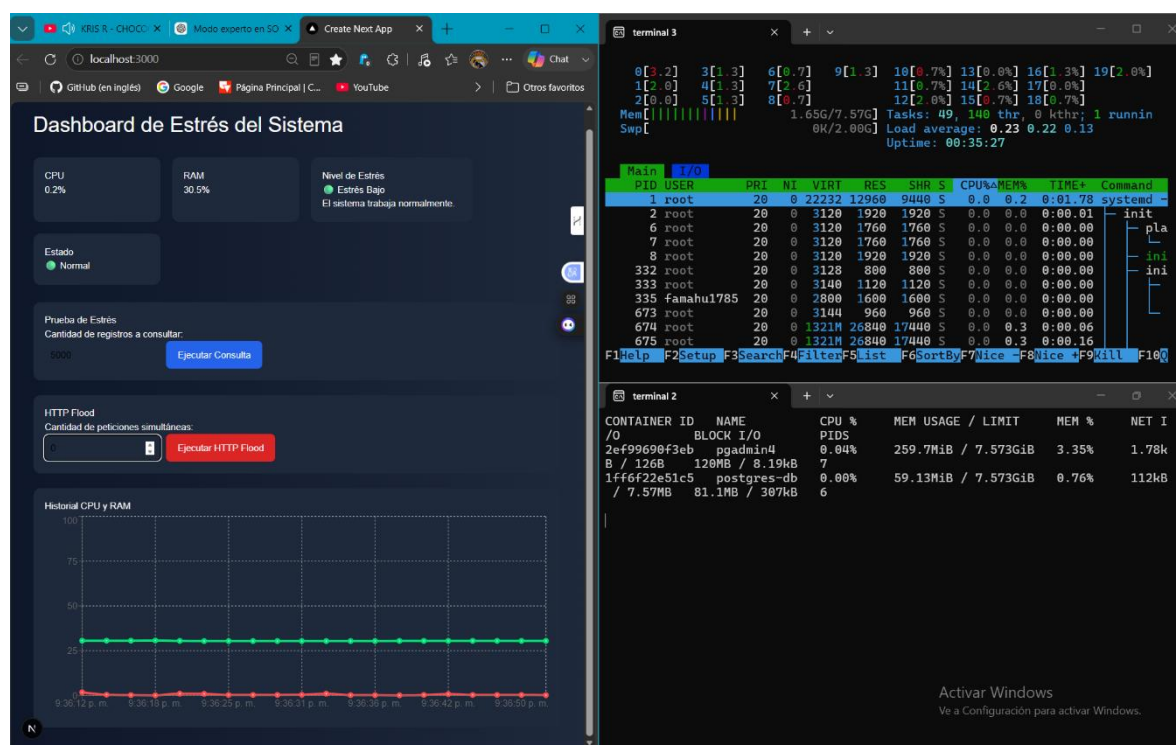
Este módulo permite enviar múltiples solicitudes simultáneas al servidor, incrementando el número de operaciones concurrentes procesadas por la aplicación y por PostgreSQL.

La cantidad de peticiones puede configurarse dinámicamente desde la interfaz web, permitiendo evaluar el comportamiento del sistema ante distintos niveles de carga.

6. Prueba HTTP Flood con 200 Peticiones

6.1 Estado previo a la ejecución

Figura 2. Estado previo a la ejecución del HTTP Flood.

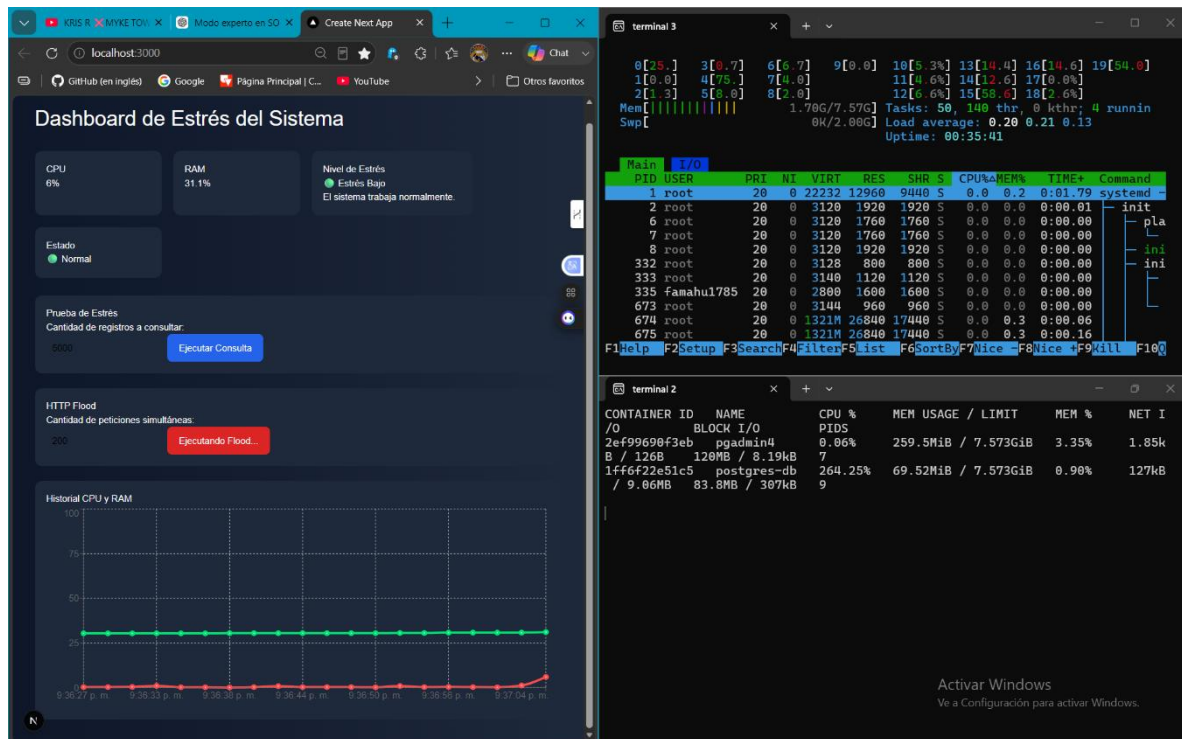


Análisis

Los indicadores muestran una utilización muy baja de CPU y un consumo estable de memoria RAM. El dashboard clasifica el estado como Estrés Bajo y PostgreSQL permanece prácticamente inactivo. Estas condiciones permiten garantizar que los resultados observados posteriormente son consecuencia directa de la carga generada.

6.2 Ejecución de 200 peticiones concurrentes

Figura 3. Ejecución de HTTP Flood con 200 peticiones concurrentes.



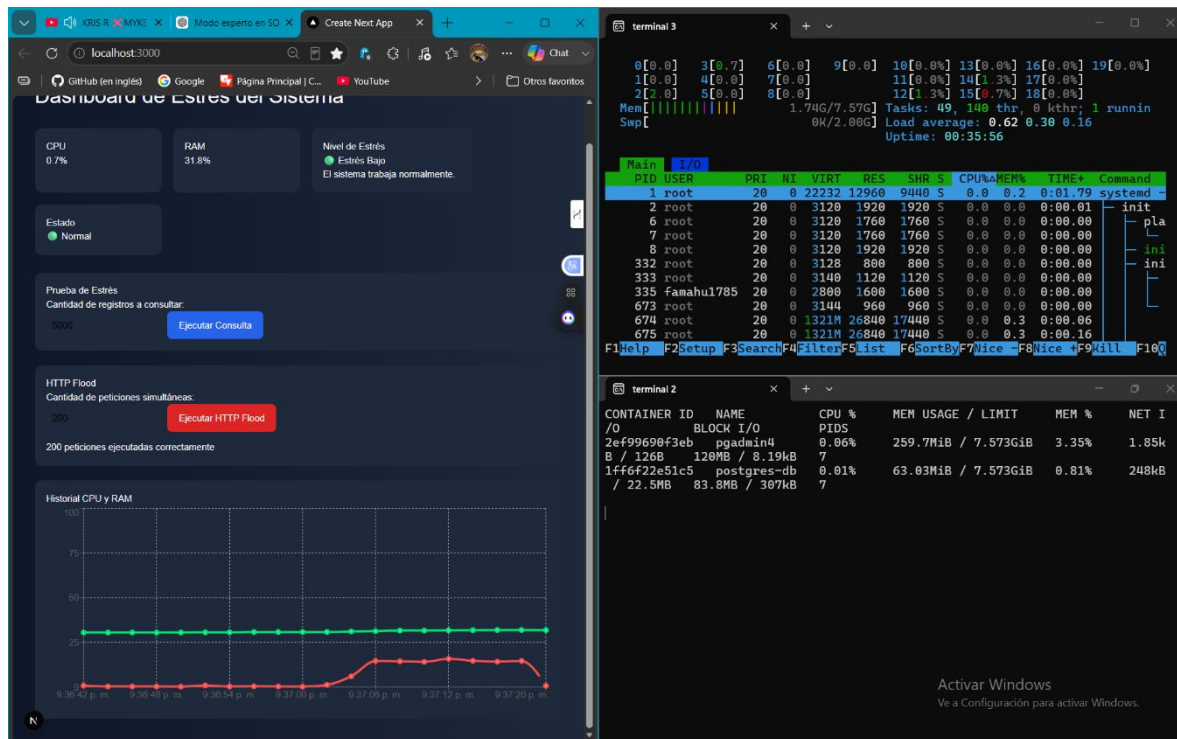
Análisis

Durante la ejecución se observó un incremento significativo en el uso de CPU. Varios núcleos alcanzaron porcentajes superiores al 50%, mientras que PostgreSQL registró aproximadamente un 264% de utilización de CPU según Docker Stats. Asimismo, aumentó el número de procesos activos y la actividad general del sistema.

Los resultados demuestran que incluso una carga relativamente moderada puede producir incrementos visibles en el consumo de recursos.

6.3 Recuperación posterior

Figura 4. Recuperación posterior al HTTP Flood de 200 peticiones.



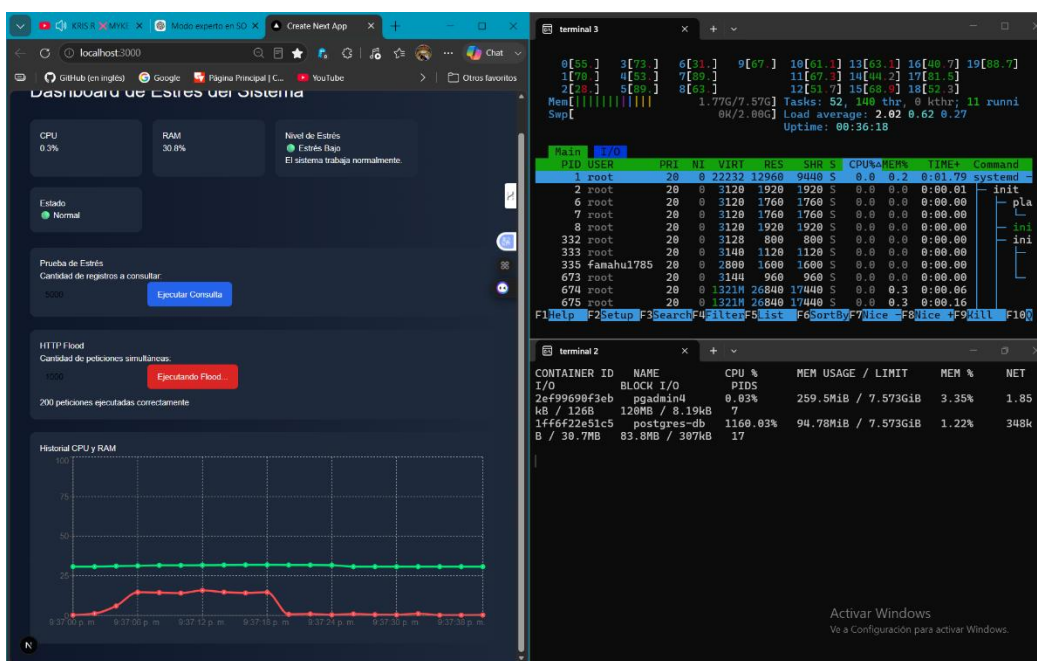
Análisis

Los recursos regresaron rápidamente a niveles cercanos a los observados inicialmente. Los núcleos del procesador disminuyeron su actividad, PostgreSQL redujo drásticamente su consumo de CPU y el número de procesos activos volvió a valores normales. Esto evidencia una adecuada capacidad de recuperación frente a cargas moderadas.

7. Prueba HTTP Flood con 1000 Peticiones

7.1 Pico máximo de carga

Figura 5. Ejecución de HTTP Flood con 1000 peticiones concurrentes.



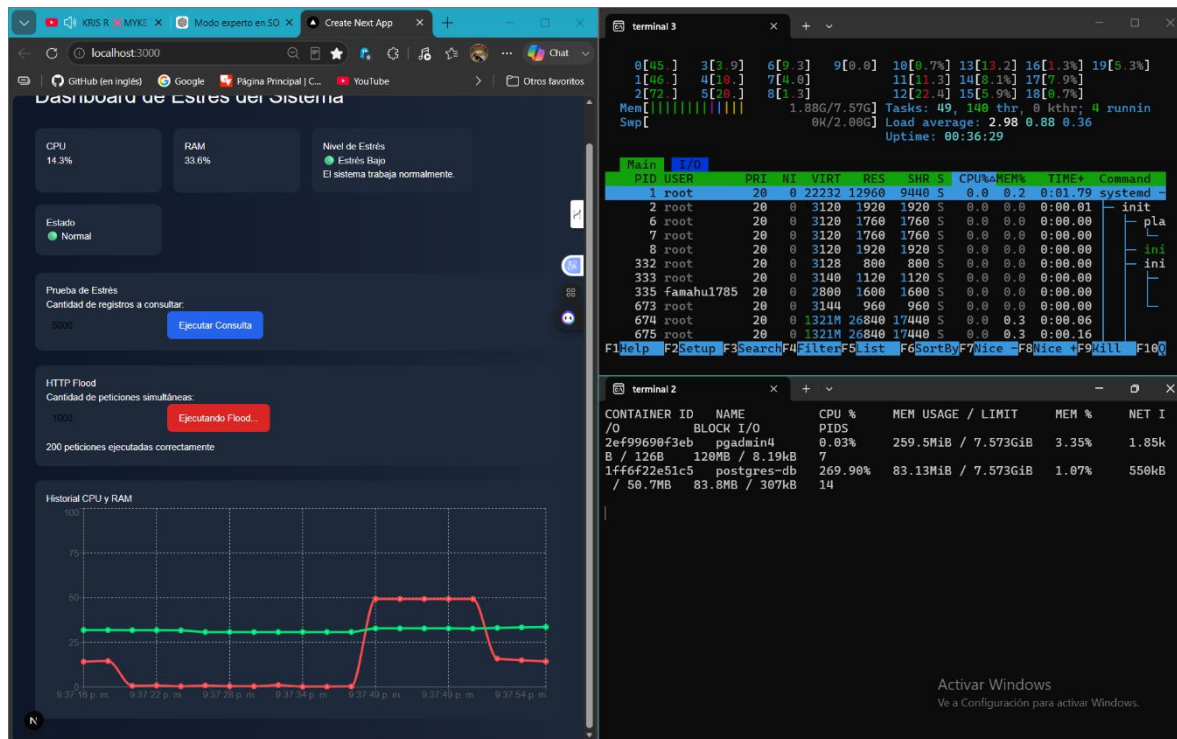
Análisis

Esta fue la prueba más exigente desarrollada durante el proyecto. Se observó un uso intensivo de múltiples núcleos del procesador, alcanzando valores cercanos al 90% en varios hilos de ejecución. El Load Average aumentó hasta 2.02 y PostgreSQL registró aproximadamente 1160% de utilización de CPU debido al procesamiento simultáneo de múltiples solicitudes.

Además, se incrementó el consumo de memoria y el número de procesos activos, evidenciando una elevada carga sobre la infraestructura.

7.2 Proceso de estabilización

Figura 6. Recuperación posterior al HTTP Flood de 1000 peticiones.



Análisis

Tras finalizar la ejecución, el sistema comenzó un proceso gradual de estabilización. El uso de CPU descendió progresivamente, disminuyó el consumo de memoria y se redujo el número de procesos activos. Los resultados demuestran que la arquitectura implementada es capaz de recuperarse adecuadamente incluso después de escenarios de estrés considerable.

8. Verificación del Endpoint HTTP Flood

Figura 7. Ejecución manual del endpoint HTTP Flood.

```
famahu1785@Famahu:~/proyecto-so/stress-dashboard$ curl http://localhost:3000/api/http-flood
{"success":true,"requests":50,"message":"HTTP Flood ejecutado"}famahu1785@Famahu:~/proyecto-so/stress-dashboard$
```


Análisis

La respuesta obtenida confirma que el endpoint fue ejecutado correctamente y que el servidor procesó las solicitudes sin presentar errores. Esto permitió verificar el correcto funcionamiento del mecanismo de generación de carga.

9. Registro de Eventos en Base de Datos

9.1 Inserción de registros

Figura 8. Inserción de registros mediante insert-flood.

```
famahu1785@Famahu:/mnt/c/Users/ASUS$ curl "http://localhost:3000/api/insert-flood?amount=10"
{"success":true,"inserted":10}famahu1785@Famahu:/mnt/c/Users/ASUS$ SELECT CO
```

Análisis

La respuesta obtenida confirma que la aplicación logró insertar correctamente la cantidad solicitada de registros dentro de la tabla logs_carga.

9.2 Verificación de registros almacenados

Figura 9. Conteo de registros almacenados.

```
famahu1785@Famahu:/mnt/c/Users/ASUS$ docker exec -it postgres-db psql -U adm
in -d appdb
psql (16.14 (Debian 16.14-1.pgdg13+1))
Type "help" for help.

appdb=# SELECT COUNT(*) FROM logs_carga;
 count
-----
      10
(1 row)

appdb=# |
```

Análisis

La consulta COUNT(*) confirmó que la información fue almacenada exitosamente dentro de la base de datos.

9.3 Consulta de eventos registrados

Figura 10. Registros almacenados en logs_carga.

```
appdb=# SELECT * FROM logs_carga LIMIT 5;
 id |      tipo      | mensaje |      created_at
-----+-----+-----+-----
  1 | INSERT_FLOOD | Registro 0 | 2026-06-12 03:04:53.130229
  2 | INSERT_FLOOD | Registro 1 | 2026-06-12 03:04:53.135751
  3 | INSERT_FLOOD | Registro 2 | 2026-06-12 03:04:53.137114
  4 | INSERT_FLOOD | Registro 3 | 2026-06-12 03:04:53.138304
  5 | INSERT_FLOOD | Registro 4 | 2026-06-12 03:04:53.139465
(5 rows)
```

Análisis

Los registros evidencian la ejecución de diferentes pruebas realizadas durante el proyecto. Es posible observar eventos asociados a cargas de 50, 200 y 1000 peticiones, además de inserciones manuales. Esta información proporciona un historial completo de las pruebas ejecutadas.

10. Simulación de Lock Contention

10.1 Apertura de una transacción

Figura 11. Inicio de transacción con recurso bloqueado.

```
famahu1785@Famahu:~/proyecto-so/stress-dashboard$ docker exec -it postgres-d
b psql -U admin -d appdb
psql (16.14 (Debian 16.14-1.pgdg13+1))
Type "help" for help.

appdb=# SELECT * FROM productos LIMIT 1;
 id | nombre | precio |      created_at
-----+-----+-----+-----
  1 | Producto 1 | 590.67 | 2026-06-03 13:42:34.299958
(1 row)

appdb=# BEGIN;
BEGIN
appdb=# UPDATE productos
SET nombre = 'PRODUCTO_BLOQUEADO'
WHERE id = 1;
UPDATE 1
appdb=# |
```

Análisis

Se inició una transacción utilizando BEGIN y posteriormente se ejecutó una actualización sobre una fila específica. Debido a que la transacción permaneció abierta, PostgreSQL mantuvo bloqueado el recurso correspondiente.

10.2 Bloqueo de una segunda transacción

Figura 12. Segunda transacción bloqueada.

```
appdb=# UPDATE productos
SET nombre = 'INTENTO_MODIFICACION'
WHERE id = 1;
|
```

Análisis

La segunda consulta quedó en espera debido a que intentaba acceder al mismo recurso que permanecía bloqueado por la transacción anterior. Este comportamiento constituye un ejemplo clásico de Lock Contention.

10.3 Verificación mediante pg_stat_activity

Figura 13. Identificación del bloqueo mediante pg_stat_activity.

```
pid | state | query
-----+-----+-----
31  |       |
32  |       |
2267 | idle  | SELECT c.oid::pg_catalog.regclass, c.relkind, inhdetachpend
ing, pg_catalog.pg_get_expr(c.relpartbound, c.oid) +
| FROM pg_catalog.pg_class c, pg_catalog.pg_inherits i
| WHERE c.oid = i.inhrelid AND i.inhparent = '24612'
| ORDER BY pg_catalog.pg_get_expr(c.relpartbound, c.oid) = 'D
EFAULT', c.oid::pg_catalog.regclass::pg_catalog.text;
2285 | idle  | UPDATE productos
| SET nombre = 'INTENTO_MODIFICACION'
| WHERE id = 1;
2310 | idle  | COMMIT;
2326 | active | SELECT pid, state, query
| FROM pg_stat_activity;
28  |
27  |
30  |
(9 rows)
```

Activar Windows
Ve a Configuración para activar Windows.

Análisis

La vista `pg_stat_activity` permitió identificar claramente el proceso bloqueador y el proceso bloqueado. Las columnas `wait_event_type` y `wait_event` mostraron los valores `Lock` y `transactionid` respectivamente, confirmando que PostgreSQL detectó correctamente la situación de contención.

10.4 Liberación del bloqueo

Figura 14. Ejecución de COMMIT.

```
appdb=# COMMIT;  
COMMIT  
appdb=# |
```

Figura 15. Finalización exitosa de la consulta bloqueada.

```
appdb=# UPDATE productos  
SET nombre = 'INTENTO_MODIFICACION'  
WHERE id = 1;  
UPDATE 1  
appdb=# |
```

Análisis

Una vez ejecutado `COMMIT`, PostgreSQL liberó el recurso bloqueado permitiendo que la segunda transacción continuara normalmente. Esto demuestra el correcto funcionamiento de los mecanismos de sincronización implementados por el motor de base de datos.

11. Validación Final

Figura 16. Verificación final de registros.

```
famahu1785@Famahu:/mnt/c/Users/ASUS$ docker exec -it postgres-db psql -U adm
in -d appdb
psql (16.14 (Debian 16.14-1.pgdg13+1))
Type "help" for help.

appdb=# SELECT COUNT(*) FROM logs_carga;
 count
-----
      10
(1 row)

appdb=# |
```

Análisis

La consulta final confirmó que la información almacenada permaneció consistente después de todas las pruebas realizadas, demostrando la integridad de los datos incluso bajo condiciones de carga y concurrencia.

12. Nuevas Pruebas de Estrés y Monitoreo con IA

Durante la ampliación del proyecto se incorporó un dashboard de monitoreo capaz de visualizar métricas de CPU, RAM y estado general del sistema en tiempo real.

También se realizaron pruebas de consulta masiva y entrenamiento de modelos de inteligencia artificial para evaluar el comportamiento de la infraestructura bajo cargas adicionales.

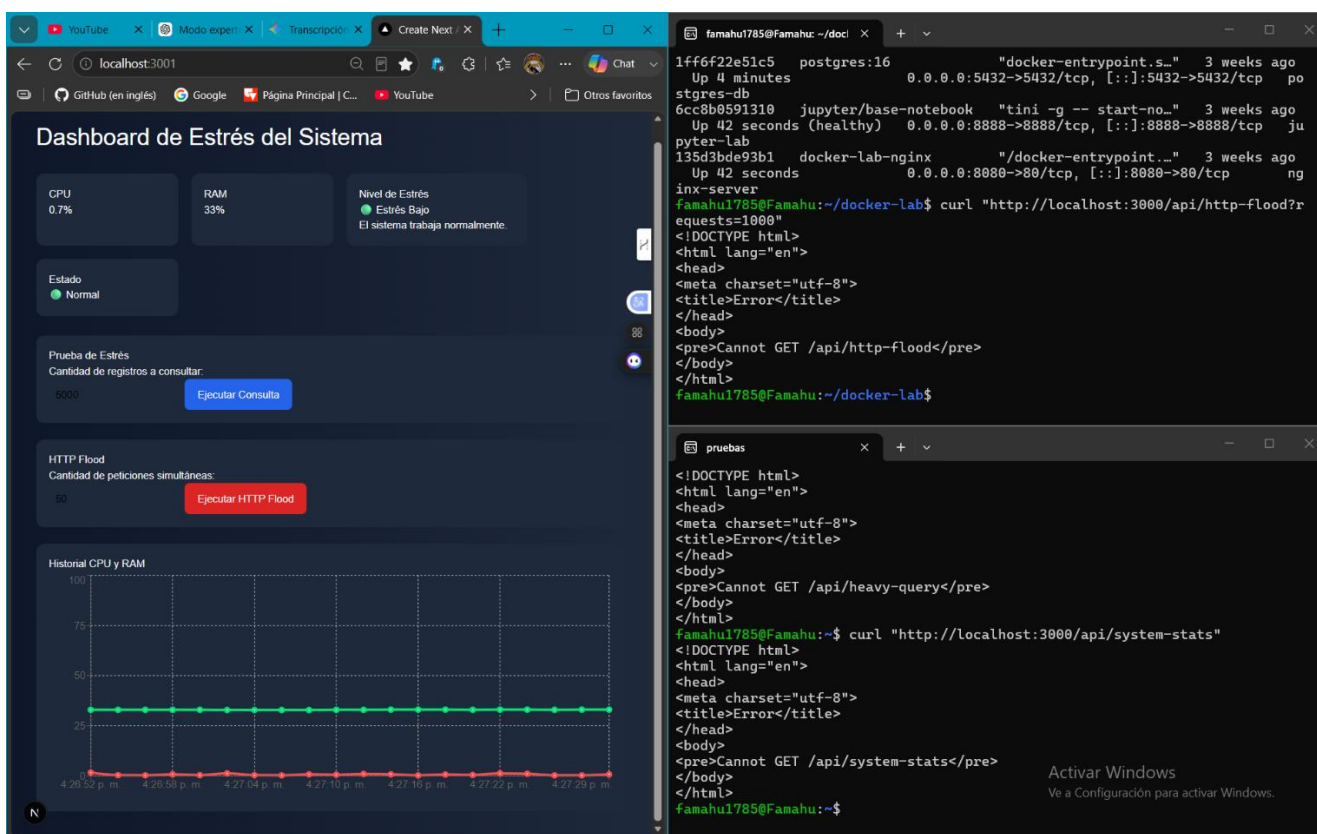
Figura 17. Dashboard en estado normal.



Análisis

Se observa que el sistema reporta Estrés Bajo, con utilización reducida de CPU y memoria RAM, representando el comportamiento normal de operación.

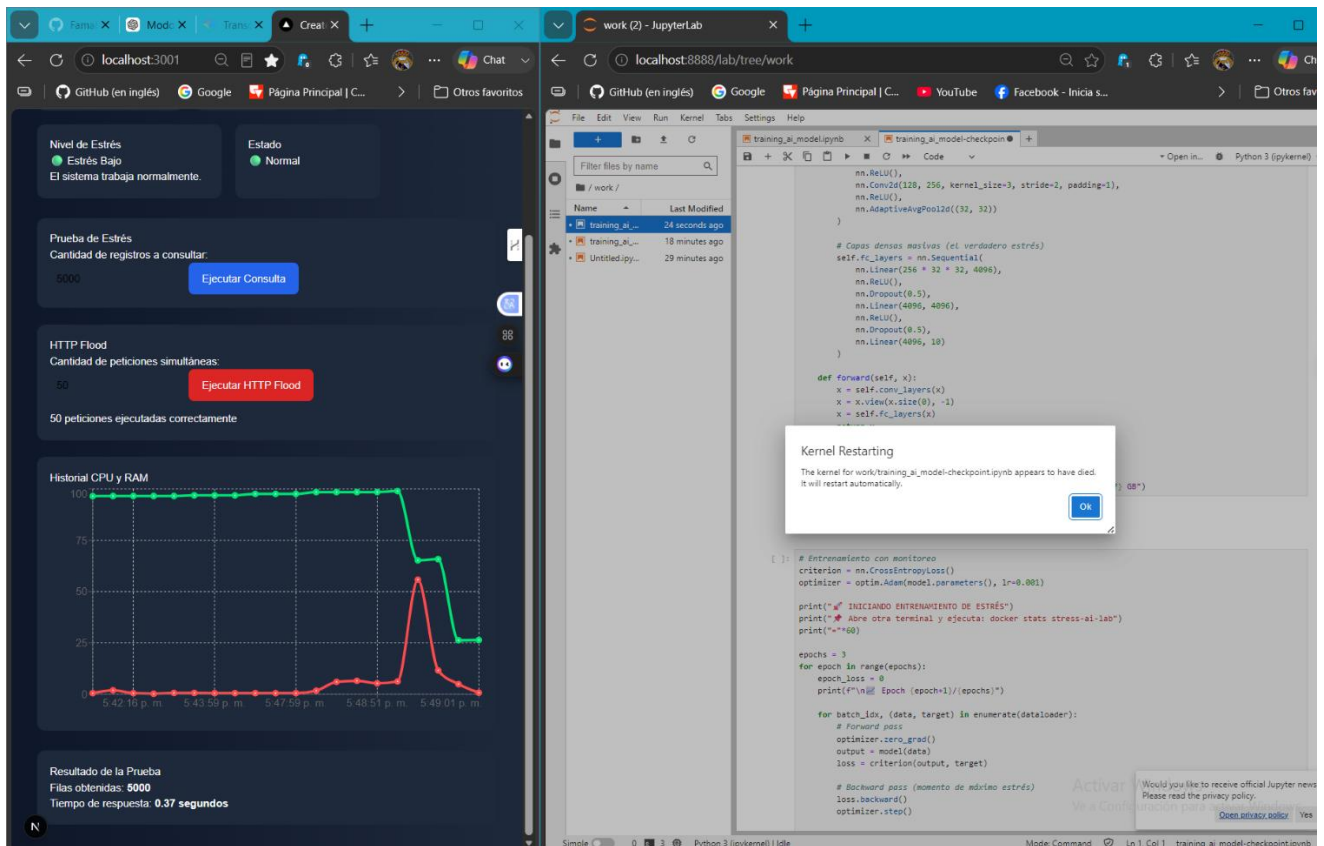
Figura 18. Prueba de estrés con consulta de 5000 registros.



Análisis

La prueba permitió recuperar 5000 registros en aproximadamente 0.37 segundos. Durante la ejecución se observó un incremento considerable en el uso de memoria RAM.

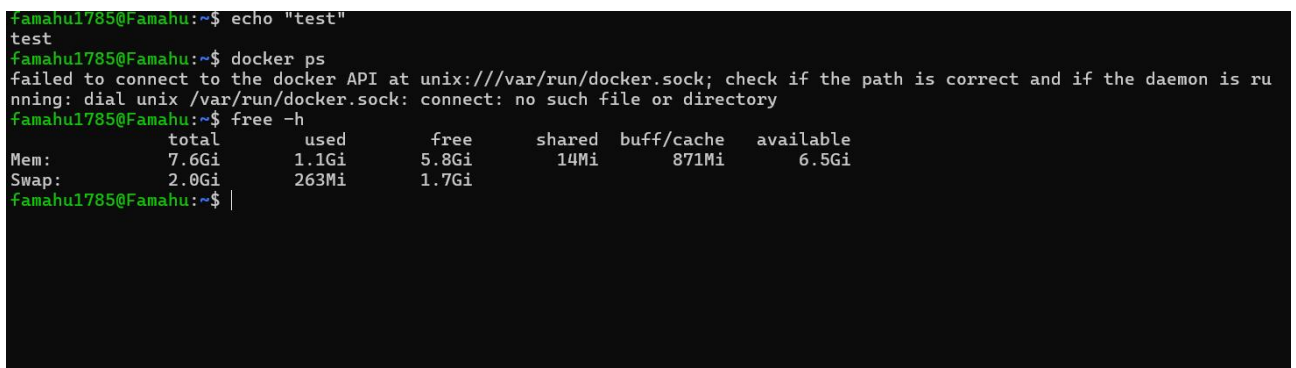
Figura 19. Reinicio automático del Kernel durante la prueba.



Análisis

Durante el entrenamiento del modelo de IA y la ejecución simultánea de consultas masivas, el kernel de JupyterLab finalizó inesperadamente debido al agotamiento de recursos. El sistema reinició automáticamente el proceso, evidenciando una situación de sobrecarga de memoria.

Figura 20. Resultado de Echo Test.



Análisis

La prueba Echo permitió verificar la comunicación correcta entre cliente y servidor, validando el funcionamiento de los endpoints implementados.

Figura 21. Evidencia A.

```
appdb=# SELECT COUNT(*) FROM transacciones;
count
-----
520000
(1 row)
```

Figura 22. Evidencia B.

The screenshot displays two terminal windows from a user named 'famahu1785' on a machine named 'Famahu'.

The left window shows the result of a SQL query: `SELECT COUNT(*) FROM transacciones;`. The output is a single row with the value 520000.

The right window shows a system status overview, including a table of running processes and system statistics.

Main	I/O	PID	USER	PRI	NI	VIRT	RES	SHR	S	CPU%	MEM%	TIME+	Command
		1	root	20	0	21748	11980	8940	S	0.0	0.2	0:01.08	init
		189	messagebus	20	0	9632	4800	4320	S	0.7	0.1	0:00.33	@dbus-
		2	root	20	0	3120	1920	1920	S	0.0	0.0	0:00.00	init
		6	root	20	0	3120	1760	1760	S	0.0	0.0	0:00.00	pla
		7	root	20	0	3120	1760	1760	S	0.0	0.0	0:00.00	ini
		8	root	20	0	3120	1920	1920	S	0.0	0.0	0:00.00	ini
		333	root	20	0	3128	800	800	S	0.0	0.0	0:00.00	
		335	root	20	0	3140	960	960	S	0.0	0.0	0:00.00	
		340	famahu1785	20	0	2800	1440	1440	S	0.0	0.0	0:00.00	
		602	root	20	0	3144	1120	1120	S	0.0	0.0	0:00.01	
		603	root	20	0	1321M	29984	17920	S	0.0	0.4	0:00.55	
		604	root	20	0	1321M	29984	17920	S	0.0	0.4	0:00.97	
		605	root	20	0	1321M	29984	17920	S	0.0	0.4	0:00.00	
		606	root	20	0	1321M	29984	17920	S	0.0	0.4	0:00.00	
		607	root	20	0	1321M	29984	17920	S	0.0	0.4	0:00.54	
		608	root	20	0	1321M	29984	17920	S	0.0	0.4	0:00.00	
		609	root	20	0	1321M	29984	17920	S	0.0	0.4	0:00.22	
		610	root	20	0	1321M	29984	17920	S	0.0	0.4	0:00.50	
		611	root	20	0	1321M	29984	17920	S	0.0	0.4	0:00.68	
		612	root	20	0	1321M	29984	17920	S	0.0	0.4	0:00.02	
		613	root	20	0	1321M	29984	17920	S	0.0	0.4	0:00.16	
		614	root	20	0	1321M	29984	17920	S	0.0	0.4	0:00.37	
		615	root	20	0	1321M	29984	17920	S	0.0	0.4	0:00.61	
		648	root	20	0	1321M	29984	17920	S	0.0	0.4	0:00.56	
		3138	root	20	0	1321M	29984	17920	S	0.0	0.4	0:00.19	
		337	root	20	0	6656	4000	3520	S	0.0	0.1	0:00.01	log
		417	famahu1785	20	0	6072	4960	3360	S	0.0	0.1	0:00.01	log
		445	root	20	0	6668	4000	3520	S	0.0	0.1	0:00.00	ini
		500	root	20	0	6072	5120	3520	S	0.0	0.1	0:00.01	
		572	root	20	0	3124	800	800	S	0.0	0.0	0:00.00	
		573	root	20	0	3140	960	960	S	0.0	0.0	0:00.00	
		574	famahu1785	20	0	6204	5280	3520	S	0.0	0.1	0:00.03	
		695	famahu1785	20	0	557M	125M	18560	S	0.0	1.6	0:06.43	
		729	famahu1785	20	0	557M	125M	18560	S	0.0	1.6	0:00.04	
		730	famahu1785	20	0	557M	125M	18560	S	0.0	1.6	0:00.00	
		758	famahu1785	20	0	4241M	1717M	151M	Active	100.0	100.0	0:15.86	
		761	famahu1785	20	0	4241M	1717M	151M	Se a	100.0	100.0	0:00.00	
		762	famahu1785	20	0	4241M	1717M	151M	S	0.0	22.2	0:00.09	Windows

System statistics shown at the top of the right window:

- Mem: 3.89G/7.57G
- Tasks: 49, 162 thr, 0 kthr, 1 runnin
- Load average: 0.09 0.11 0.74
- Uptime: 01:05:04

Figura 23. Evidencia C.

```
famahu1785@famahu:~$ docker exec -it postgres-db psql -U admin -d appdb
psql (16.14 (Debian 16.14-1.pgdg13+1))
Type "help" for help.

appdb=# SELECT pid,
        now() - query_start AS duration,
        state,
        LEFT(query,80)
FROM pg_stat_activity
WHERE state != 'idle'
ORDER BY duration DESC;
 pid | duration | state | left
-----+-----+-----+-----
 1572 | 00:00:00 | active | SELECT pid,
        now() - query_start AS duration,
        state,
        LEFT(qu
(1 row)

appdb=# |
```

Análisis

Las evidencias adicionales muestran diferentes estados operativos del sistema durante las fases de prueba y validación, complementando los resultados obtenidos en las pruebas principales.

Figura 24. Consulta mediante SELECT y monitoreo de estado.

```
famahu1785@famahu:~$ docker exec -it postgres-db psql -U admin -d appdb
psql (16.14 (Debian 16.14-1.pgdg13+1))
Type "help" for help.

appdb=# SELECT state, count(*)
FROM pg_stat_activity
GROUP BY state;
 state | count
-----+-----
 active |      5
  idle  |      2
(3 rows)

appdb=# |
```

Análisis

La consulta permitió verificar el estado interno de los registros almacenados y confirmar la correcta interacción entre la aplicación y PostgreSQL.

13. Conclusiones

- Se implementó exitosamente un módulo HTTP Flood capaz de generar distintos niveles de carga concurrente sobre el sistema.
 - PostgreSQL respondió adecuadamente ante escenarios de estrés moderado y alto.
 - La prueba de 1000 peticiones concurrentes permitió observar un comportamiento realista de consumo intensivo de CPU.
 - Docker Stats y htop resultaron herramientas fundamentales para el monitoreo.
 - La tabla logs_carga permitió mantener un registro histórico de las ejecuciones realizadas.
 - Se comprobó experimentalmente el funcionamiento de los mecanismos de bloqueo y control de concurrencia implementados por PostgreSQL.
 - La vista pg_stat_activity permitió identificar de forma precisa situaciones de contención.
 - Las pruebas adicionales con consultas masivas y entrenamiento de IA evidenciaron el impacto del consumo de memoria RAM sobre la estabilidad del sistema.
 - El proyecto permitió aplicar conceptos fundamentales de Sistemas Operativos relacionados con procesos, concurrencia, sincronización, monitoreo y administración eficiente de recursos.
-

14. Referencias

- [1] PostgreSQL Global Development Group, PostgreSQL Documentation.
- [2] Docker Inc., Docker Documentation.
- [3] Vercel, Next.js Documentation.
- [4] React Team, React Documentation.
- [5] PgAdmin Development Team, PgAdmin Documentation.
- [6] A. Silberschatz, P. Galvin y G. Gagne, Operating System Concepts, Wiley.